Discovering Customer Segments and Product Associations — a KDD Project

Author: Meron Tesfandrias · **Course:** CMPE 255 · **Repo:** MeronDT/CMPE-255-Data-Mining-process · **App:** `<deployed URL>`

Data: *Online Retail II* — UCI Machine Learning Repository (dataset 502, CC BY 4.0), donated by D. Chen. Transaction lines of a UK-based online retailer of giftware (many customers are wholesalers), 1 Dec 2009 – 9 Dec 2011: 1,067,371 lines in the two Excel sheets, of which 22,523 are repeated across the sheets, so 1,044,848 distinct lines.

KDD (Fayyad, Piatetsky-Shapiro & Smyth, 1996) is "the nontrivial process of identifying **valid, novel, potentially useful, and ultimately understandable** patterns in data". It defines nine steps, usually grouped into five stages:

Stage	Fayyad's steps	In this notebook
0 · Domain & goals	1. Understand the application domain, prior knowledge and the end user's goals	business context, two discovery goals, evaluation plan and pass marks
1 · **Selection**	2. Create a target data set	load both sheets once each, define equal 12-month discovery and validation periods
2 · **Preprocessing**	3. Data cleaning and preprocessing	cancellations (matched one-to-one within each period), non-product codes, invalid prices, duplicates, missing customers
3 · **Transformation**	4. Data reduction and projection	customer RFM table (log, scaling, PCA); invoice × product basket matrix
4 · **Data mining**	5. Match goals to a mining method · 6. Choose algorithms · 7. Mine	K-means segmentation · FP-Growth association rules
5 · **Interpretation / Evaluation**	8. Interpret mined patterns	profile segments and rules; test them on the **next year's** data against baselines
→ Act	9. Act on the discovered knowledge	consolidated outputs + a web app for marketing staff

Headline results:

- **Segments.** Four customer segments that are stable (out-of-bag bootstrap ARI 0.92) and separate next year's behaviour sharply. The top 17 % of customers ("Champions") produced 64 % of revenue, and 96 % of them bought again the following year, against 40 % of "Lapsed one-off buyers". But they are a useful *partition*, not natural clusters, and a plain RFM score ranks next-year activity better

(AUC 0.765 vs 0.739, gap CI +0.018 to +0.034). So the segments are delivered as explainable *labels*, with the RFM score beside them as the ranking tool.

- **Rules.** 325 associations (598 directional rules). **59 %** of the rules meet all three thresholds (support, confidence, lift) again in the following year, against 12 % for rules just below the thresholds. The stable ones are mostly *variants* of the same product. The few genuinely **cross-product** associations, *matching-design sets* such as red spotty paper napkins + cups → plates (84 % confidence, 90 % the next year), are the novel part, but only about a quarter of them hold up the next year.
- **Delivery.** Both are delivered in a web app for marketing staff, which ships exactly the segmentation that was validated.

How to run: *Runtime → Run all* (≈ 4–7 min on free Colab; most of it is downloading and reading the 44 MB Excel file once — later runs use a cache). Every figure, table and the app are written to `artifacts/`.

⚙️ Setup & configuration

In [1]:
```
try:
    import mlxtend, openpyxl
except ImportError:   # pinned: newer mlxtend releases force-upgrade pandas/scik
    %pip install -q "mlxtend==0.23.4" openpyxl
import json, platform, re, time, urllib.request, warnings, zipfile
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn, mlxtend

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth",
plt.rcParams["figure.figsize"] = (8, 4.5)
print(f"python {platform.python_version()} | pandas {pd.__version__} | scikit-le
assert tuple(int(x) for x in mlxtend.__version__.split(".")[:2]) >= (0, 23), "as
```

```
python 3.13.18 | pandas 3.0.5 | scikit-learn 1.9.1 | mlxtend 0.25.0
```

In [2]:
```
# ============================ CONFIGURATION ============================
DATA_URL = "https://archive.ics.uci.edu/static/public/502/online+retail+ii.zip"
DISCOVERY_END  = pd.Timestamp("2010-12-01")   # mine patterns on Dec 2009 - Nov
VALIDATION_END = pd.Timestamp("2011-12-01")   # ... and check them on Dec 2010 -
K_RANGE        = range(3, 7)    # candidate numbers of customer segments (fewer
MIN_SUPPORT    = 0.01           # itemset must appear in >= 1 % of invoices
MIN_CONFIDENCE = 0.40           # P(consequent | antecedent)
MIN_LIFT       = 3.0            # >= 3x more likely than by chance
RANDOM_STATE   = 42
# Success criteria for the validity tests in Stage 5. Set in revision 2 (critiqu
# the corrected cleaning; the first run's results were known, so they are docume
CRITERIA = {"segments_beat_rfm_score_auc": True,   # clusters must rank next-yea
            "rules_hold_all_thresholds": 0.50}     # >= 50 % of rules meet suppo
# ======================================================================

ART = Path("artifacts"); DATA, FIG, REP, APP = (ART / p for p in ("data", "figur
for p in (DATA, FIG, REP, APP):
```

```
    p.mkdir(parents=True, exist_ok=True)

def savefig(name):
    plt.tight_layout(); plt.savefig(FIG / f"{name}.png", dpi=120, bbox_inches="t
```

Stage 0 — Understanding the application domain and the KDD goals

Fayyad step 1: learn the domain and the relevant prior knowledge, and identify the end user's goals.

Domain and prior knowledge

The data come from a UK-based, store-less online retailer selling mainly unique all-occasion giftware (tableware, Christmas decorations, bags, children's items). Many customers are **wholesalers** who resell the products, so order sizes vary enormously, and sales are strongly **seasonal**, peaking in September–November ahead of Christmas. Chen, Sain & Guo (2012) first analysed the 2010-11 part of this data with RFM-based segmentation, which is the established retail practice:

- **RFM** (recency, frequency, monetary value) is the standard, explainable way to group customers by purchasing behaviour; recent, frequent, high-spending customers are the most valuable and most likely to buy again. A simple **RFM score** is therefore the baseline the segments must be compared with.
- **Market-basket analysis** (Agrawal et al., 1993) finds products bought together; it underpins "customers also bought" recommendations and store/catalogue layout. The retailer designs its products in **collections** (e.g. "red retrospot"), so matching items bought together are partly expected.

End-user goals → discovery goals

End user	Business goal	KDD goal (descriptive, not predictive)
Marketing manager	target campaigns at the right customers	**G1** — discover **customer segments** from purchasing behaviour that each call for a different action
Merchandising / web team	increase basket size with cross-selling	**G2** — discover **product associations** ("customers who buy X also buy Y")

How the patterns will be evaluated (Fayyad's four criteria)

Criterion	Test used here	Pass mark (set in revision 2)
Valid	patterns found in the discovery year must still hold in the **following year**. Segments must separate next-year activity and spend;	segments rank next-year activity at least as well as an RFM score; ≥ 50 % of rules

Criterion	Test used here	Pass mark (set in revision 2)
	rules must meet support, confidence and lift again	meet all three thresholds next year, clearly more than near-miss rules
Novel	rules are split into obvious *variants* (same product, other colour/design) and genuinely **cross-product** associations, and the split is audited by hand; findings are compared with what the merchandiser already knows (the collections)	— (argued, not scored)
Potentially useful	each segment gets a concrete action and the revenue at stake; validated rules feed a recommendation tool; how many products and baskets the rules reach	—
Understandable	segments described in plain words against customer tertiles; rules shown as readable product names	—

The pass marks were added in revision 2, before the re-run with the corrected cleaning but after the first run's results were known. They are documented targets, not blind ones.

Stage 1 — Selection

Fayyad step 2: select the data set, or subset of variables and records, on which discovery will be performed.

In [3]:
```
cache = DATA / "online_retail_II_v2.pkl"
if cache.exists():
    raw = pd.read_pickle(cache)
else:
    t0 = time.time()
    archive = DATA / "online_retail_ii.zip"
    if not archive.exists():
        urllib.request.urlretrieve(DATA_URL, archive)
    with zipfile.ZipFile(archive) as z:
        xlsx = [n for n in z.namelist() if n.endswith(".xlsx")][0]
        z.extract(xlsx, DATA)
    sheets = pd.read_excel(DATA / xlsx, sheet_name=None)          # BOTH sheets
    print({name: len(s) for name, s in sheets.items()})
    raw = pd.concat([s.assign(sheet=name) for name, s in sheets.items()], ignore
    # The sheets overlap: "Year 2009-2010" runs to 9 Dec 2010 and "Year 2010-201
    overlap = (raw["sheet"] == "Year 2009-2010") & (pd.to_datetime(raw["InvoiceD
    print(f"Dropping {overlap.sum():,} lines of the 2009-10 sheet that the 2010-
    raw = raw[~overlap].drop(columns="sheet").reset_index(drop=True)
    raw = raw.rename(columns={"InvoiceNo": "Invoice", "UnitPrice": "Price", "Cus
    raw["Invoice"] = raw["Invoice"].astype(str)
    raw["StockCode"] = raw["StockCode"].astype(str).str.strip().str.upper()
    raw["Description"] = raw["Description"].astype(str).str.strip()
    raw["Customer ID"] = raw["Customer ID"].astype("Int64").astype(str).replace(
    raw["InvoiceDate"] = pd.to_datetime(raw["InvoiceDate"])
```

```
    raw.to_pickle(cache)
    print(f"Read Excel in {time.time() - t0:.0f}s (cached for next time)")
print(f"{len(raw):,} transaction lines x {raw.shape[1]} attributes | {raw['Invoi
raw.head()
```

```
{'Year 2009-2010': 525461, 'Year 2010-2011': 541910}
Dropping 22,523 lines of the 2009-10 sheet that the 2010-11 sheet repeats (1-9 De
c 2010)
Read Excel in 134s (cached for next time)
1,044,848 transaction lines x 8 attributes | 2009-12-01 07:45:00 -> 2011-12-09 1
2:50:00
```

Out[3]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085	United Kingdom

In [4]:

```
overview = pd.DataFrame({
    "lines": [len(raw)], "invoices": [raw["Invoice"].nunique()], "customers": [r
    "products": [raw["StockCode"].nunique()], "countries": [raw["Country"].nuniq
    "cancellation_lines": [raw["Invoice"].str.startswith("C").sum()],
    "lines_without_customer_%": [round(raw["Customer ID"].isna().mean() * 100, 1
display(overview)
monthly = raw[~raw["Invoice"].str.startswith("C")].assign(revenue=lambda d: d["Q
            .set_index("InvoiceDate").resample("MS")["revenue"].sum() / 1e3
fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(monthly.index[:-1], monthly.to_numpy()[:-1], marker="o", c="C0"); ax.set
ax.plot(monthly.index[-1:], monthly.to_numpy()[-1:], "o", mfc="white", c="C0")
ax.annotate("Dec 2011: only 9 days", (monthly.index[-1], monthly.iloc[-1]), xyte
for x, lab in [(DISCOVERY_END, "  discovery | validation"), (VALIDATION_END, "
    ax.axvline(x, c="r", ls="--"); ax.text(x, ax.get_ylim()[1] * .95, lab, color
savefig("selection_monthly_revenue")
```

	lines	invoices	customers	products	countries	cancellation_lines	lines_without_cus
0	1044848	53628	5942	5131	43	19165	



Target data set

Decision	Choice	Rationale
Records	every transaction line once: the 2009-10 sheet runs to 9 Dec 2010 and the 2010-11 sheet starts on 1 Dec 2010, so the **22,523 repeated lines** (1–9 Dec 2010) are dropped at load time, before any cleaning	full history, no double counting
Attributes	invoice, product code, quantity, date, price and customer for RFM and baskets; description as the product label; country descriptive only	every attribute has a role
Discovery period	1 Dec 2009 – 30 Nov 2010 (12 months)	one full seasonal cycle to mine patterns
Validation period	1 Dec 2010 – 30 Nov 2011 (12 months)	an equally long, untouched year to test whether patterns are **valid**. The last 9 days (1–9 Dec 2011, the partial month on the chart) are used only for the deployed app
Segmentation scope	lines with a Customer ID	RFM needs a customer
Basket scope	all invoices (with or without Customer ID)	an anonymous basket is still a basket. Anonymous invoices are larger (2.4), so 5.3 checks that they do not drive the rules

No information crosses the split. Every data-dependent cleaning rule (matching cancellations to purchases) runs within each period, so nothing from the validation year changes the discovery year.

🔍 Critic review — Selection

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude

sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Selection** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the target data set matches the KDD goals, whether the discovery/validation split is justified, and whether the scoping decisions (e.g. anonymous transactions) are explained. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	23 % of lines have no Customer ID and are excluded from segmentation; discovery/validation periods must each cover a season.	Stated the limitation (those lines are still used for baskets); confirmed both periods span a full Dec–Nov cycle.
2	Cleaning ran across the discovery/validation split; validation year 9 days longer; no pass marks. (C+ overall)	Matching within each period; equal 12-month windows; pass marks (disclosed as set late); anonymous baskets profiled.
3	The two Excel sheets overlap (22,523 lines loaded twice); revenue chart markers off the plot. (B− overall)	Overlap dropped at load time; chart redrawn on real dates.

Stage 2 — Preprocessing

Fayyad step 3: basic operations such as removing noise, deciding how to handle missing data, and accounting for known changes.

In [5]:
```
df = raw.copy(); log = []
def step(action, mask_removed):
    global df
    log.append({"action": action, "lines_removed": int(mask_removed.sum()), "lin
    df = df[~mask_removed]
period_of = lambda d: np.select([d["InvoiceDate"] < DISCOVERY_END, d["InvoiceDat

is_cancel = df["Invoice"].str.startswith("C")
cancellations = df[is_cancel].copy()
step("Separate cancellation lines (invoice starts with 'C')", is_cancel)

# Remove purchases that were cancelled in full: each cancellation is matched to
# purchase by the same customer of the same product and quantity, within the SAM
# the discovery/validation boundary). Unmatched cancellations (partial returns)
pur = df[df["Customer ID"].notna()][["Customer ID", "StockCode", "Quantity", "In
```

```
can = cancellations[cancellations["Customer ID"].notna()][["Customer ID", "Stock
          .assign(Quantity=lambda d: -d["Quantity"], cid=lambda d: d.index)
pur["period"], can["period"] = period_of(pur), period_of(can)
match = pd.merge_asof(can.sort_values("InvoiceDate"), pur.sort_values("InvoiceDa
                      by=["Customer ID", "StockCode", "Quantity", "period"], dir
match = match.drop_duplicates("pid")                                    # one pu
cancellations["matched"] = cancellations.index.isin(match["cid"])
print(f"{len(can):,} cancellation lines with a customer: {cancellations['matched
      f"{(~cancellations['matched'] & cancellations['Customer ID'].notna()).sum(
step("Remove purchases cancelled in full (one-to-one, earlier purchase, same per

non_product = ~df["StockCode"].str.match(r"^\d{5}")          # POST, DOT, M, BANK
print("Non-product codes removed:", sorted(df.loc[non_product, "StockCode"].uniq
step("Remove non-product codes (postage, fees, manual adjustments, tests)", non_
step("Remove lines with quantity <= 0 or price <= 0", (df["Quantity"] <= 0) | (d
dupes = df.duplicated()
print(f"Exact duplicates: {dupes.sum():,} lines = {(df.loc[dupes, 'Quantity'] *
      f"{df.loc[dupes, 'Invoice'].nunique():,} invoices affected")
step("Remove exact duplicate lines (same invoice, product, quantity, price and s

# One description per product (codes have spelling variants / damaged-stock note
df["Description"] = df["Description"].replace("nan", np.nan)
canonical = df.groupby("StockCode")["Description"].agg(lambda s: s.value_counts(
df["Description"] = df["StockCode"].map(canonical)
df["Revenue"] = df["Quantity"] * df["Price"]
df["period"] = period_of(df)

# Returns for the return_rate feature: product-code cancellations that did NOT c
returns = cancellations[~cancellations["matched"] & cancellations["StockCode"].s
returns["value"] = -returns["Quantity"] * returns["Price"]; returns["period"] =

preprocessing_log = pd.DataFrame(log); preprocessing_log.to_csv(REP / "preproces
display(preprocessing_log)
print(f"Clean: {len(df):,} lines, {df['Invoice'].nunique():,} invoices, {df['Sto
      f"{df['Customer ID'].nunique():,} customers; {df['Customer ID'].isna().mea
```

```
18,446 cancellation lines with a customer: 5,825 matched one-to-one to an earlier
purchase, 12,621 unmatched (partial returns, adjustments, older purchases)
Non-product codes removed: ['ADJUST', 'ADJUST2', 'AMAZONFEE', 'B', 'BANK CHARGE
S', 'C2', 'C3', 'D', 'DCGS0003', 'DCGS0004', 'DCGS0006', 'DCGS0016', 'DCGS0027',
'DCGS0036', 'DCGS0037', 'DCGS0039', 'DCGS0041', 'DCGS0044', 'DCGS0053', 'DCGS005
5', 'DCGS0056', 'DCGS0057', 'DCGS0058', 'DCGS0059', 'DCGS0060'] ...
Exact duplicates: 11,622 lines = 0.3% of revenue; 4,285 invoices affected
```

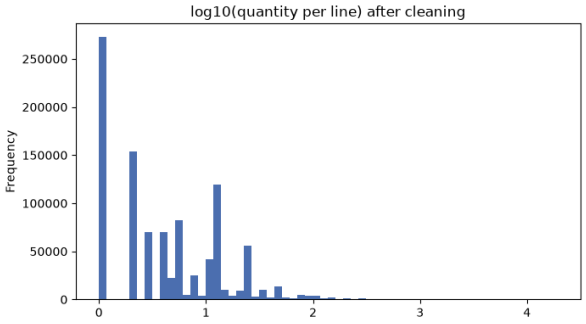
	action	lines_removed	lines_left
0	Separate cancellation lines (invoice starts with 'C')	19165	1025683
1	Remove purchases cancelled in full (one-to-one, earlier ...	5825	1019858
2	Remove non-product codes (postage, fees, manual adjustme...	4661	1015197
3	Remove lines with quantity <= 0 or price <= 0	5937	1009260
4	Remove exact duplicate lines (same invoice, product, qua...	11622	997638

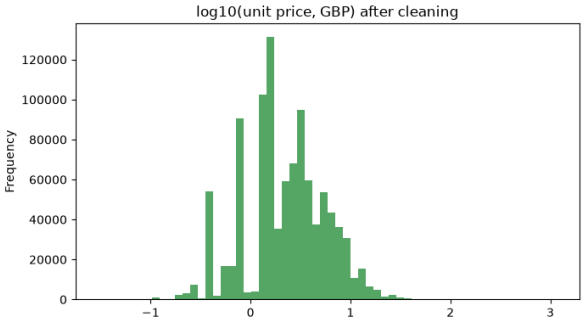
```
Clean: 997,638 lines, 39,266 invoices, 4,699 products, 5,842 customers; 22.7% of
lines have no customer
```

In [6]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
np.log10(df["Quantity"]).plot.hist(bins=60, ax=axes[0], color="#4C72B0", title="
np.log10(df["Price"]).plot.hist(bins=60, ax=axes[1], color="#55A868", title="log
```

```
savefig("preprocessing_distributions")
print("Largest lines by revenue:"); display(df.nlargest(5, "Revenue")[["Invoice"
print("Highest unit prices:"); display(df.drop_duplicates("StockCode").nlargest(

# Anonymous baskets: do invoices without a Customer ID look like the others? (th
inv = df.groupby("Invoice").agg(has_id=("Customer ID", lambda s: s.notna().any()
anon = inv.groupby("has_id").agg(invoices=("lines", "size"), median_lines=("line
                                 mean_lines=("lines", "mean"), median_revenue=("
display(anon.round(1))
```





Largest lines by revenue:

	Invoice	InvoiceDate	Customer ID	Description	Quantity	Price	Revenue
432176	530715	2010-11-04 11:36:00	15838	ROTATING SILVER ANGELS T-LIGHT HLDR	9360	1.69	15818.40
228042	511465	2010-06-08 12:59:00	18008	PINK PAPER PARASOL	3500	2.55	8925.00
851263	567423	2011-09-20 11:05:00	17450	SET OF TEA COFFEE SUGAR TINS PANTRY	1412	5.06	7144.72
663484	550461	2011-04-18 13:20:00	15749	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2.10	6539.40
461644	533027	2010-11-15 16:02:00	NaN	PAPER CHAIN KIT 50'S CHRISTMAS	835	6.95	5803.25

Highest unit prices:

	StockCode	Description	Price	Quantity
268525	22656	VINTAGE BLUE KITCHEN CABINET	295.0	1
268526	22655	VINTAGE RED KITCHEN CABINET	295.0	1
528106	22826	LOVE SEAT ANTIQUE WHITE METAL	175.0	2
406342	22827	RUSTIC SEVENTEEN DRAWER SIDEBOARD	165.0	1
406996	22828	REGENCY MIRROR WITH SHUTTERS	145.0	2

has_id	invoices	median_lines	p99_lines	mean_lines	median_revenue
no Customer ID	2922	19.5	565.5	77.3	183.8
with Customer ID	36344	15.0	101.6	20.9	301.3

Preprocessing decisions

- **Cancellations (19,165 lines) are separated, not netted.** A cancellation is a different event from a purchase, and netting would hide it. Each cancellation is matched to **one** purchase: the most recent *earlier* purchase by the same customer of the same product and quantity, **within the same period**. That removes 5,825 purchases cancelled in full. (The first version matched across periods, ignored time order and removed every matching purchase: 13,653 lines.) The 12,621 unmatched cancellations (partial returns, or cancellations of older purchases) feed `return_rate`. It uses product codes only, excludes the fully cancelled purchases, and is measured as returned value ÷ gross product spend.
- **Non-product codes (4,661 lines)** — postage (`POST` , `DOT`), bank charges, Amazon fees, manual adjustments (`M` , `ADJUST`), samples, tests and `DCGS…` gift codes — are not products and would create meaningless rules ("postage → postage").
- **Zero/negative quantity or price (5,937 lines)** are stock corrections, damaged/lost goods and free items, not sales.
- **Exact duplicates (11,622 lines, 0.3 % of revenue, on 4,285 invoices)** — identical invoice, product, quantity, price and second — are treated as double entries. Baskets are binary, so rules are unaffected. (Before the sheet overlap was removed at load time, this step also caught the 22k repeated lines, which made duplicates look three times more common than they are.)
- **Missing Customer ID (22.7 % of clean lines):** kept for basket analysis, excluded from segmentation (RFM needs a customer). Anonymous invoices are **larger** (mean 77 products vs 21; 99th percentile 566 vs 102) and cheaper per invoice (median £184 vs £301), probably trade or batch orders. 5.3 re-mines the rules without them.
- **Extreme lines:** the largest lines (e.g. 9,360 rotating silver-angel T-light holders for £15,818) look like genuine wholesale orders, and the highest unit prices are real furniture (a vintage kitchen cabinet at £295). They are kept, because the log transformation in Stage 3 limits their influence on clustering.

Result: 997,638 clean lines, 39,266 invoices, 4,699 products, 5,842 identified customers.

🔍 Critic review — Preprocessing

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md` , `round_2.md` , `round_3.md` .

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Preprocessing** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether every cleaning rule is justified by an observed data problem, how missing customer IDs and cancellations are handled, and whether any rule could silently bias the segments or rules. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Cancellations matched only on exact customer/product/quantity; are the 33k duplicate lines really errors?	Partial returns captured by `return_rate`; duplicates identical to the second, baskets binary so rules unaffected — documented.
2	Cancellation matching many-to-one, ignored time; return_rate double-counted; duplicates unexamined.	One-to-one, time-ordered, within-period matching (5,825 lines); return_rate rebuilt; duplicate revenue share reported.
3	Most "duplicates" were the sheet overlap.	With the overlap removed, true duplicates are 11,622 lines (0.3 % of revenue).

Stage 3 — Transformation

Fayyad step 4: find useful features to represent the data, using dimensionality reduction or transformation methods.

3.1 Customer table (for G1): RFM and behavioural features

In [7]:
```
def customer_features(lines, returns, snapshot):
    """One row per customer, measured at `snapshot` (the day after the period en
    c = lines.dropna(subset=["Customer ID"]).groupby("Customer ID")
    f = pd.DataFrame({
        "recency_days": (snapshot - c["InvoiceDate"].max()).dt.days,
        "frequency": c["Invoice"].nunique(),
        "monetary": c["Revenue"].sum(),
        "tenure_days": (snapshot - c["InvoiceDate"].min()).dt.days,
        "distinct_products": c["StockCode"].nunique(),
        "country": c["Country"].agg(lambda s: s.mode()[0]),
    })
    f["avg_basket_value"] = f["monetary"] / f["frequency"]
    returned = returns.groupby("Customer ID")["value"].sum().reindex(f.index).fi
    f["return_rate"] = returned / (f["monetary"] + returned)      # share of gro
```

```
    return f

disc_lines = df[df["period"] == "discovery"]
val_lines = df[df["period"] == "validation"]
customers = customer_features(disc_lines, returns[returns["period"] == "discover
print(f"{len(customers):,} customers in the discovery period"); display(customer
```

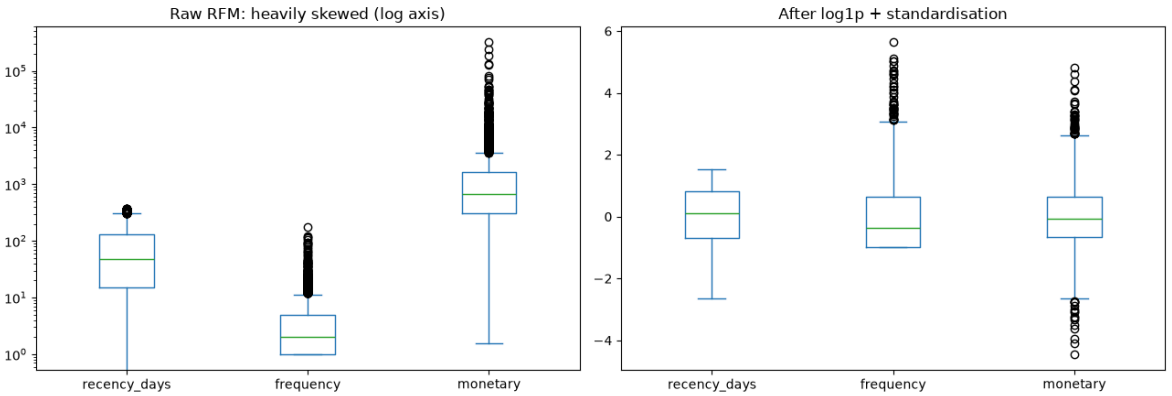
```
4,230 customers in the discovery period
```

	count	mean	std	min	25%	50%	75%	max
recency_days	4230.0	86.60	94.62	0.00	15.00	47.00	131.75	364.00
frequency	4230.0	4.27	7.46	1.00	1.00	2.00	5.00	174.00
monetary	4230.0	1934.10	8357.84	1.55	304.96	669.44	1637.26	319471.34
tenure_days	4230.0	217.71	117.39	0.00	113.00	244.00	320.00	364.00
distinct_products	4230.0	62.67	84.05	1.00	17.00	38.00	78.00	1707.00
avg_basket_value	4230.0	375.27	504.35	1.55	177.98	285.59	421.54	11880.84
return_rate	4230.0	0.01	0.04	0.00	0.00	0.00	0.00	0.76

In [8]:
```
from sklearn.preprocessing import FunctionTransformer, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA

SEGMENT_FEATURES = ["recency_days", "frequency", "monetary"]          # classic
projector = Pipeline([("log", FunctionTransformer(np.log1p)), ("scale", Standard
Z = projector.fit_transform(customers[SEGMENT_FEATURES])

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
customers[SEGMENT_FEATURES].plot.box(ax=axes[0], logy=True, title="Raw RFM: heav
pd.DataFrame(Z, columns=SEGMENT_FEATURES).plot.box(ax=axes[1], title="After log1
savefig("transformation_rfm")
print("Correlation of transformed RFM features:"); display(pd.DataFrame(Z, colum
pca = PCA(2).fit(Z); print(f"2-D PCA projection keeps {pca.explained_variance_ra
```




```
Correlation of transformed RFM features:
```

	recency_days	frequency	monetary
recency_days	1.00	-0.48	-0.42
frequency	-0.48	1.00	0.81
monetary	-0.42	0.81	1.00

```
2-D PCA projection keeps 94% of the variance (used for plots)
```

Why these transformations. Frequency and monetary value span several orders of magnitude (1 to 174 orders; £2 to £319k per customer) and are right-skewed, while K-means uses Euclidean distance, so a handful of wholesalers would dominate untransformed data. `log1p` compresses the scale and standardisation gives each RFM dimension equal weight. Only three features are used so each segment can be explained in one sentence.

Caveat, tested in 5.2: frequency and monetary still correlate at 0.81 after transformation, so purchase *volume* is effectively weighted twice. 5.2 clusters an alternative representation (recency + one averaged volume factor). It agrees with the main segmentation at ARI 0.73 and predicts next year no better, so RFM is kept. Recency's log scale stretches the first weeks, but the tertile labels and next-year results show that cooling and lapsed customers are still separated. Wholesalers are not modelled separately: customers with an average basket above £1,000 are only 0.7–8.4 % of each segment (5.1). The 2-D PCA projection keeps 94 % of the variance, so the plots in Stage 5 are faithful.

3.2 Basket matrix (for G2): invoice × product

In [9]:
```
item_name = df.drop_duplicates("StockCode").set_index("StockCode")["Description"
dup = item_name.duplicated(keep=False)
item_name[dup] = item_name[dup] + " (" + item_name[dup].index + ")"        # kee

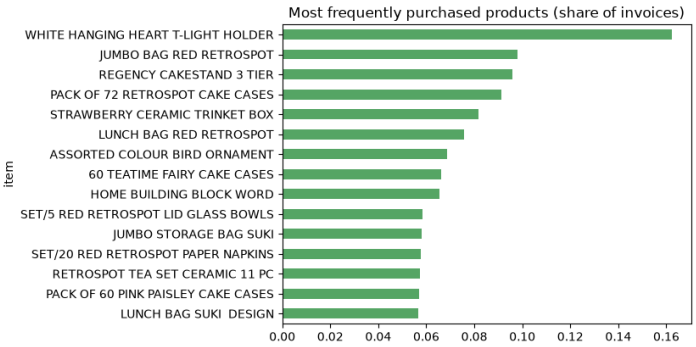
def item_support(lines):
    """Share of invoices that contain each product."""
    l = lines.assign(item=lines["StockCode"].map(item_name))
    return (l.groupby("item")["Invoice"].nunique() / l["Invoice"].nunique()).sor

def basket_matrix(lines, items):
    """Boolean invoice x product matrix restricted to `items`; keeps EVERY invoi
    l = lines.assign(item=lines["StockCode"].map(item_name))
    b = l[l["item"].isin(items)].groupby(["Invoice", "item"]).size().unstack(fil
    return b.reindex(index=l["Invoice"].unique(), columns=items, fill_value=Fals

disc_support = item_support(disc_lines)
frequent_items = disc_support[disc_support >= MIN_SUPPORT].index
disc_baskets = basket_matrix(disc_lines, frequent_items)
basket_sizes = disc_lines.groupby("Invoice")["StockCode"].nunique()
print(f"{disc_lines['Invoice'].nunique():,} invoices x {disc_lines['StockCode'].
      f"with support >= {MIN_SUPPORT:.0%}; matrix {disc_baskets.shape[0]:,} x {d
print("Distinct products per invoice:", basket_sizes.describe(percentiles=[.5, .

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
disc_support.head(15).sort_values().plot.barh(ax=axes[0], color="#55A868", title
axes[1].loglog(np.arange(1, len(disc_support) + 1), disc_support.to_numpy(), c="
axes[1].axhline(MIN_SUPPORT, c="r", ls="--", label=f"min support {MIN_SUPPORT:.0
axes[1].set(title="Long tail: support of every product, by rank", xlabel="produc
savefig("transformation_item_support")
```

```
19,616 invoices x 4,047 products -> reduced to 714 products with support >= 1%; m
atrix 19,616 x 714 (14 MB)
Distinct products per invoice: {'50%': 15.0, '90%': 49.0, '99%': 177.0, 'max': 57
7.0}
```



Why this reduction is safe. An itemset can only be frequent if every item in it is frequent (the *Apriori property*). Removing the 3,333 products that appear in fewer than 1 % of invoices therefore loses no possible rule at this support level. It shrinks the matrix from 4,047 to 714 products (14 MB), which makes FP-Growth run in seconds. Every invoice is kept, so support is measured against all 19,616 discovery-year baskets. The long-tail chart shows the cost: most of the catalogue can never appear in a rule at 1 % support. 5.3 measures that coverage and tries other thresholds. A typical invoice has 15 distinct products (90th percentile 49, 99th 177).

🔍 Critic review — Transformation

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Transformation** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the chosen representations suit each mining goal, whether scaling and reduction choices are justified, and whether information that matters was lost. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Frequency and monetary correlate 0.81 after transformation, double-weighting volume.	Kept RFM for explainability; recorded the caveat and a PCA/drop-one alternative as a next iteration.
2	F/M redundancy left in; item-support figure corrupted; basket sizes not shown.	Alternative recency + volume clustering tested (no better); figure fixed with a long-tail plot; basket-size distribution.
3	Recency scaling never varied.	Not changed (documented).

Stage 4 — Data mining

Fayyad steps 5–7: match the goals to a mining method, choose the algorithms and parameters, and search for patterns.

Goal	Method (step 5)	Algorithm (step 6)	Why
G1 segments	clustering	**K-means** on log-RFM	fast, simple to explain, centroids are readable profiles. *k* is chosen with silhouette, plus out-of-bag bootstrap stability for every *k*
G2 associations	association-rule mining	**FP-Growth** + rule generation	same output as Apriori but without candidate generation — much faster on wide baskets. Thresholds: 1 % support, 40 % confidence, lift 3. 5.3 tests the alternatives

Choosing *k*. Silhouette is highest at *k* = 2 (0.41), but two groups only split active from inactive customers, which is too coarse to act on. Within 3–6, *k* = 4 has the best silhouette (0.34) and the best out-of-bag stability (ARI 0.92, worst resample 0.85); from *k* = 5 the worst resample drops to 0.55. A silhouette of 0.34 and the PCA plot (one continuous cloud) show these are **a useful partition of a continuum, not natural clusters**. G1 is worded accordingly.

4.1 Customer segmentation (step 7, G1)

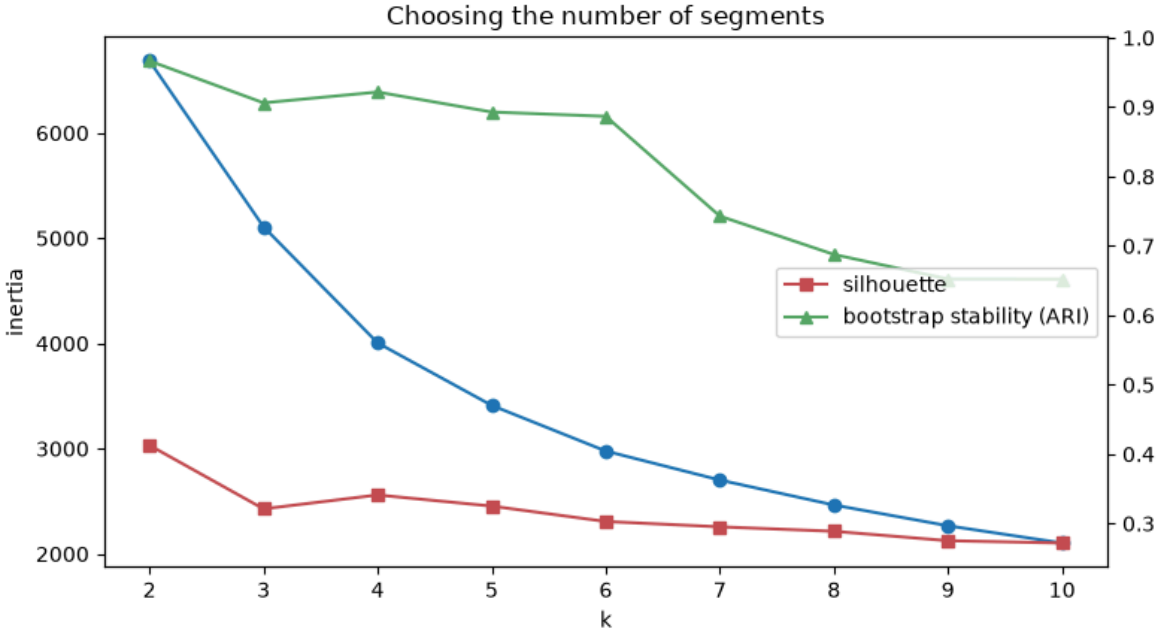
In [10]:
```
from sklearn.cluster import KMeans
from sklearn.base import clone
from sklearn.metrics import silhouette_score, adjusted_rand_score

k_search = []
for k in range(2, 11):
    km = KMeans(k, n_init=10, random_state=RANDOM_STATE).fit(Z)
    k_search.append({"k": k, "inertia": km.inertia_, "silhouette": silhouette_sc
k_search = pd.DataFrame(k_search)

# Stability of each k: refit on bootstrap resamples and compare labels on the OU
def oob_stability(k, n_boot=20):
    base = KMeans(k, n_init=10, random_state=RANDOM_STATE).fit(Z).labels_
    rng, aris = np.random.default_rng(RANDOM_STATE), []
    for b in range(n_boot):
        inb = rng.integers(0, len(Z), len(Z)); oob = np.setdiff1d(np.arange(len(
        aris.append(adjusted_rand_score(base[oob], KMeans(k, n_init=5, random_st
    return np.mean(aris), np.min(aris)
k_search[["oob_ari_mean", "oob_ari_min"]] = [oob_stability(k) for k in k_search[
k_search = k_search.round(4); k_search.to_csv(REP / "k_selection.csv", index=Fal
fig, ax = plt.subplots(); ax.plot(k_search["k"], k_search["inertia"], marker="o"
ax2 = ax.twinx(); ax2.plot(k_search["k"], k_search["silhouette"], marker="s", c=
ax2.plot(k_search["k"], k_search["oob_ari_mean"], marker="^", c="#55A868", label
ax.set_title("Choosing the number of segments"); savefig("mining_k_selection")
```

```
K = int(k_search[k_search["k"].isin(K_RANGE)].sort_values("silhouette", ascendin
print(f"Chosen k = {K} (best silhouette within {list(K_RANGE)}; k = 2 scores hig
```

	k	inertia	silhouette	oob_ari_mean	oob_ari_min
0	2	6684.9717	0.4127	0.9667	0.9230
1	3	5108.0676	0.3211	0.9062	0.8251
2	4	4004.7341	0.3411	0.9218	0.8451
3	5	3411.1565	0.3251	0.8930	0.5477
4	6	2977.2673	0.3029	0.8868	0.6874
5	7	2701.6478	0.2952	0.7430	0.3756
6	8	2464.5305	0.2888	0.6877	0.3565
7	9	2267.1617	0.2752	0.6523	0.4538
8	10	2103.4211	0.2719	0.6522	0.4325




```
Chosen k = 4 (best silhouette within [3, 4, 5, 6]; k = 2 scores higher but only s
plits active from inactive customers)
```

In [11]:
```
segmenter = Pipeline([("project", clone(projector)), ("kmeans", KMeans(K, n_init
customers["cluster"] = segmenter.predict(customers[SEGMENT_FEATURES])
row = k_search.set_index("k").loc[K]
print(f"k = {K}: silhouette {row['silhouette']:.3f}, out-of-bag bootstrap stabil
```

```
k = 4: silhouette 0.341, out-of-bag bootstrap stability ARI mean 0.922 (min 0.84
5, 20 resamples)
```

4.2 Association rules (step 7, G2)

In [12]:
```
from mlxtend.frequent_patterns import fpgrowth, association_rules

def mine_rules(baskets, min_support=MIN_SUPPORT, min_conf=MIN_CONFIDENCE, min_li
    itemsets = fpgrowth(baskets, min_support=min_support, use_colnames=True, max
```

```
    r = association_rules(itemsets, num_itemsets=len(baskets), metric="confidenc
    r = r[r["lift"] >= min_lift].copy()
    r["antecedents"] = r["antecedents"].apply(lambda s: tuple(sorted(s)))
    r["consequents"] = r["consequents"].apply(lambda s: tuple(sorted(s)))
    r["itemset"] = [tuple(sorted(a + c)) for a, c in zip(r["antecedents"], r["co
    return itemsets, r.sort_values(["lift", "confidence"], ascending=False).rese

t0 = time.time()
itemsets, rules = mine_rules(disc_baskets)
print(f"{len(itemsets):,} frequent itemsets -> {len(rules):,} rules with confide
      f"({time.time() - t0:.0f}s). They come from {rules['itemset'].nunique()} d
display(rules.drop_duplicates("itemset")[["antecedents", "consequents", "support
```

```
1,402 frequent itemsets -> 598 rules with confidence >= 0.4 and lift >= 3.0 (2s).
They come from 325 distinct itemsets: most pairs appear as A->B and B->A.
```

	antecedents	consequents	support	confidence	lift
0	(CHILDS GARDEN FORK PINK,)	(CHILDS GARDEN TROWEL PINK,)	0.010	0.926	68.565
2	(CHILDS GARDEN TROWEL PINK,)	(CHILDS GARDEN TROWEL BLUE,)	0.011	0.796	61.250
4	(POPPY'S PLAYHOUSE BEDROOM,)	(POPPY'S PLAYHOUSE LIVINGROOM,)	0.010	0.718	59.667
6	(POPPY'S PLAYHOUSE LIVINGROOM,)	(POPPY'S PLAYHOUSE KITCHEN,)	0.011	0.898	57.585
8	(POPPY'S PLAYHOUSE KITCHEN,)	(POPPY'S PLAYHOUSE BEDROOM,)	0.012	0.797	55.863
10	(PINK REGENCY TEACUP AND SAUCER,)	(GREEN REGENCY TEACUP AND SAUCER,)	0.012	0.865	50.181
12	(PINK REGENCY TEACUP AND SAUCER,)	(ROSES REGENCY TEACUP AND SAUCER,)	0.011	0.797	43.792
14	(GREEN REGENCY TEACUP AND SAUCER,)	(ROSES REGENCY TEACUP AND SAUCER,)	0.014	0.784	43.080
16	(KEY FOB , BACK DOOR,)	(KEY FOB , FRONT DOOR,)	0.011	0.545	40.476
18	(COFFEE MUG DOG + BALL DESIGN,)	(COFFEE MUG CAT + BIRD DESIGN,)	0.012	0.725	36.832

🔍 Critic review — Data mining

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Data mining** stage of my retail KDD project below. Check it against everything this KDD stage*

> *requires. Focus especially on: whether each algorithm matches its goal, whether the parameter choices (k, minimum support, confidence, lift) are justified rather than arbitrary, and whether the stability of the segments was demonstrated. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Silhouette is best at k = 2; restricting to 3–6 needs a reason.	Justified on usefulness; k = 4 best within range (0.341); bootstrap ARI 0.94 shows stability.
2	k = 4 overrode silhouette's k = 2; no RFM baseline; thresholds unjustified; weak stability test; redundant rules.	Out-of-bag stability for every k; "partition, not natural clusters"; threshold grid; itemsets reported.
3	Grid judged on the validation year; rule table showed both directions.	Grid labelled sensitivity, not selection; one direction per itemset; headline counts 325 associations.

Stage 5 — Interpretation / Evaluation

Fayyad step 8: interpret the mined patterns, possibly returning to earlier steps, and judge them against the criteria set in Stage 0.

5.1 Segment profiles (understandable)

In [13]:
```
# Describe segments against customer TERTILES. Each label is the tertile that ho
# with that share shown, so a segment whose median sits just above a cut-off is
q = customers[SEGMENT_FEATURES + ["tenure_days"]].quantile([1 / 3, 2 / 3])
def tertile(col, v):
    return np.where(v <= q.loc[1 / 3, col], 0, np.where(v > q.loc[2 / 3, col], 2
NAMES = {"recency_days": ["active", "cooling", "lapsed"], "frequency": ["one-off
         "monetary": ["low-spend", "mid-spend", "high-spend"]}
def describe_members(g):
    parts = []
    for col, names in NAMES.items():
        shares = np.bincount(tertile(col, g[col]), minlength=3) / len(g)
        parts.append(f"{names[shares.argmax()]} {shares.max():.0%}")
    return " · ".join(parts + ([f"new {(g['tenure_days'] <= 90).mean():.0%}"] if

def profile_segments(c):
    p = c.groupby("cluster").agg(customers=("frequency", "size"), recency_days=(
                                 frequency=("frequency", "median"), monetary=("m
                                 avg_basket_value=("avg_basket_value", "median")
                                 return_rate=("return_rate", "mean"), non_UK_sha
                                 basket_over_1000_share=("avg_basket_value", lam
    p["revenue_share_%"] = (p["revenue"] / p["revenue"].sum() * 100).round(1)
    p["customer_share_%"] = (p["customers"] / p["customers"].sum() * 100).round(
    p["profile"] = [describe_members(c[c["cluster"] == k]) for k in p.index]
    return p
```

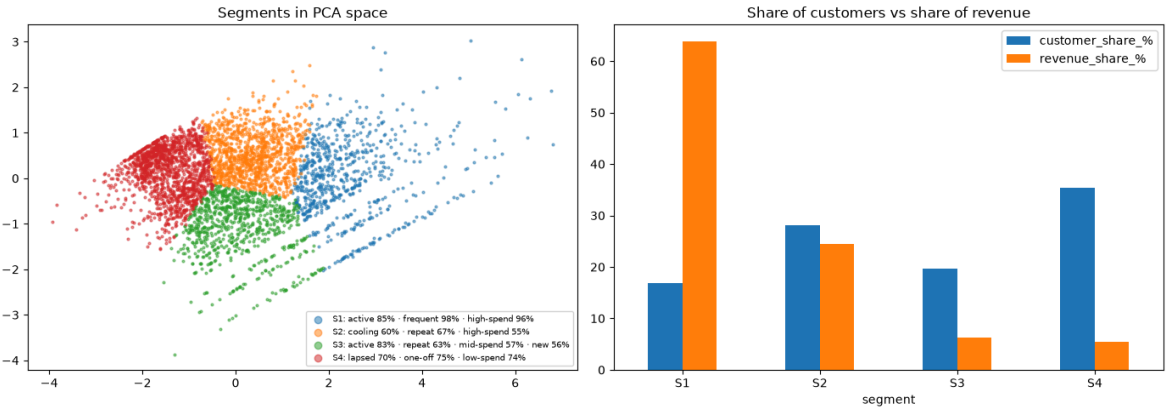
```
prof = profile_segments(customers).sort_values("monetary", ascending=False)
prof["segment"] = [f"S{i + 1}" for i in range(len(prof))]
SEGMENT_OF = prof["segment"].to_dict(); customers["segment"] = customers["cluste
prof = prof.set_index("segment").round(3); prof.to_csv(REP / "segment_profiles.c
print("Tertile cut-offs used for the labels:"); display(q.round(1))
display(prof.drop(columns="revenue"))
```

```
Tertile cut-offs used for the labels:
```

	recency_days	frequency	monetary	tenure_days
0.333333	23.0	1.0	385.6	168.0
0.666667	86.0	4.0	1230.1	295.0

segment	customers	recency_days	frequency	monetary	avg_basket_value	tenure_days
S1	712	8.0	9.0	3536.125	354.918	350.0
S2	1189	57.0	4.0	1310.340	346.490	280.0
S3	832	12.0	2.0	506.345	253.690	60.0
S4	1497	165.0	1.0	261.000	207.040	197.0

In [14]:
```
xy = PCA(2).fit(Z).transform(Z)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for s, g in customers.assign(pc1=xy[:, 0], pc2=xy[:, 1]).groupby("segment"):
    axes[0].scatter(g["pc1"], g["pc2"], s=4, alpha=.5, label=f"{s}: {prof.loc[s,
axes[0].legend(fontsize=7, markerscale=3); axes[0].set_title("Segments in PCA sp
prof[["customer_share_%", "revenue_share_%"]].plot.bar(ax=axes[1], rot=0, title=
savefig("evaluation_segments")
```



Reading the profiles (medians, discovery year). Each label is the customer tertile holding most of the segment's members, with that share. The tertile cut-offs are recency 23 / 86 days, frequency 1 / 4 orders, spend £386 / £1,230.

Segment	Who they are	Share of customers → revenue
S1 · Champions	ordered 8 days ago, 9 orders, £3,536 spent, customers for ~a year (85 % active, 98 % frequent, 96 % high-spend)	17 % → **64 %**
S2 · Loyal but cooling	4 orders and £1,310 spent, but last order 57 days ago (60 % cooling; only 55 % high-spend, so "high-spend" is a majority, not all)	28 % → 24 %
S3 · New customers	active (12 days) but only 2 months' tenure, 2 orders, £506 (56 % joined in the last 90 days)	20 % → 6 %
S4 · Lapsed one-off buyers	one order, £261, 165 days ago (70 % lapsed, 75 % one-off)	35 % → 6 %

Return rates are low and similar (0.6–1.1 % of gross spend) across segments, so returns do not distinguish them (the corrected `return_rate` no longer double-counts fully cancelled orders). Likely wholesalers (average basket > £1,000) are rare in every segment (0.7–8.4 %), and non-UK customers are 6–9 %. The actions below are written for these four groups.

In [15]:
```
# Segment names and actions, written after reading the profiles and the next-yea
SEGMENT_ACTIONS = {
    "S1": {"name": "Champions",
           "action": "Protect: account manager or VIP tier, early access to new
    "S2": {"name": "Loyal but cooling",
           "action": "Re-engage: personalised reminder with best-selling reorder
    "S3": {"name": "New customers",
           "action": "Develop: onboarding emails, cross-sell from the associatio
    "S4": {"name": "Lapsed one-off buyers",
           "action": "Low-cost win-back only (seasonal newsletter before Q4); do
}
for s in SEGMENT_ACTIONS:
    SEGMENT_ACTIONS[s]["name"] = f"{s} · {SEGMENT_ACTIONS[s]['name']} ({prof.loc
pd.DataFrame(SEGMENT_ACTIONS).T
```

Out[15]:

	name	action
S1	S1 · Champions (active 85% · frequent 98% · high-spend 96%)	Protect: account manager or VIP tier, early access to ne...
S2	S2 · Loyal but cooling (cooling 60% · repeat 67% · high-...	Re-engage: personalised reminder with best-selling reord...
S3	S3 · New customers (active 83% · repeat 63% · mid-spend ...	Develop: onboarding emails, cross-sell from the associat...
S4	S4 · Lapsed one-off buyers (lapsed 70% · one-off 75% · l...	Low-cost win-back only (seasonal newsletter before Q4); ...

5.2 Are the segments valid? Behaviour in the following year

In [16]:

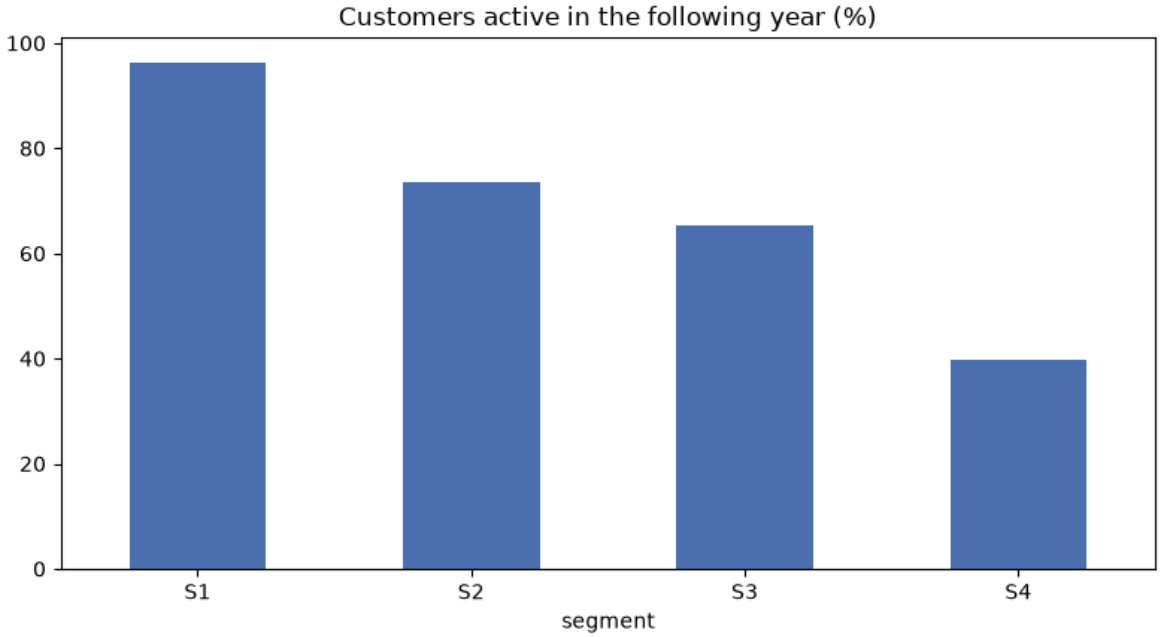
```
from sklearn.metrics import roc_auc_score
from scipy.stats import spearmanr
nxt = val_lines.dropna(subset=["Customer ID"]).groupby("Customer ID")["Revenue"]
fut = nxt.reindex(customers.index).fillna(0)
validity = customers.assign(next_year_revenue=fut).groupby("segment").agg(
    customers=("next_year_revenue", "size"),
    active_next_year_pct=("next_year_revenue", lambda s: round((s > 0).mean() *
    median_next_year_revenue=("next_year_revenue", "median"),
    total_next_year_revenue=("next_year_revenue", "sum")).reindex(prof.index)
validity["share_of_returning_customer_revenue_%"] = (validity["total_next_year_r
validity["share_of_ALL_next_year_revenue_%"] = (validity["total_next_year_revenu
validity.to_csv(REP / "segment_validity_next_year.csv"); display(validity.round(
print(f"Discovery-year customers produced {fut.sum() / val_lines['Revenue'].sum(
      f"(the rest came from new customers and anonymous invoices)")
fig, ax = plt.subplots()
validity[["active_next_year_pct"]].plot.bar(ax=ax, rot=0, legend=False, color="#
savefig("evaluation_segment_validity")

# Baselines: do the clusters predict next year better than the standard tools th
# Each segmentation is scored by how well its ORDER of groups ranks next-year ac
rank_score = lambda x: pd.Series(x, index=customers.index).rank(pct=True)
r_s, f_s, m_s = rank_score(-customers["recency_days"]), rank_score(customers["fr
rfm_score = r_s + f_s + m_s
alt = KMeans(K, n_init=20, random_state=RANDOM_STATE).fit(np.c_[Z[:, 0], Z[:, 1:
alt_rank = pd.Series(alt.labels_, index=customers.index).map(
    customers.assign(l=alt.labels_).groupby("l")["monetary"].median().rank(ascen
candidates = {
    f"K-means on log-RFM (k={K}, the project's segments)": -customers["segment"]
    "Alternative: K-means on recency + one volume factor": -alt_rank,
    "RFM score (sum of R, F, M percentile ranks), quartiles": pd.qcut(rfm_score,
    "Monetary tertiles": pd.qcut(customers["monetary"], 3, labels=False),
    "RFM score, continuous (upper bound for a score)": rfm_score,
}
active = (fut > 0).astype(int)
baseline_table = pd.DataFrame([{"segmentation": n, "groups": int(s.nunique()) if
                                "AUC_active_next_year": roc_auc_score(active, s)
                               for n, s in candidates.items()]).set_index("segme
baseline_table.to_csv(REP / "segment_baselines.csv"); display(baseline_table)
print("Agreement between the two K-means variants (ARI):", round(adjusted_rand_s
km_s, rfm_q = candidates[list(candidates)[0]].to_numpy(), candidates[list(candid
bidx = np.random.default_rng(RANDOM_STATE).integers(0, len(act), (500, len(act))
```

```
gap = [roc_auc_score(act[i], rfm_q[i]) - roc_auc_score(act[i], km_s[i]) for i in
print(f"AUC gap, RFM-score quartiles minus K-means segments: {np.mean(gap):+.3f}
```

segment	customers	active_next_year_pct	median_next_year_revenue	total_next_year_reve
S1	712	96.2	2493.0	437633
S2	1189	73.6	559.7	136665
S3	832	65.3	256.4	49352
S4	1497	39.8	0.0	35881

```
Discovery-year customers produced 70% of all next-year revenue (the rest came fro
m new customers and anonymous invoices)
```



segmentation	groups	AUC_active_next_year	spearman_next_year_revenue
K-means on log-RFM (k=4, the project's segments)	4.0	0.739	0.575
Alternative: K-means on recency + one volume factor	4.0	0.745	0.574
RFM score (sum of R, F, M percentile ranks), quartiles	4.0	0.765	0.601
Monetary tertiles	3.0	0.729	0.586
RFM score, continuous (upper bound for a score)	NaN	0.783	0.628

```
Agreement between the two K-means variants (ARI): 0.726
AUC gap, RFM-score quartiles minus K-means segments: +0.026 (95 % bootstrap CI +
0.018 to +0.034)
```

Do the segments predict next year? They were built only from 2009-10 behaviour, yet they sharply separate who buys in 2010-11:

Segment	Active next year	Median spend next year
Champions	96 %	£2,493
S2	74 %	£560
New customers	65 %	£256
Lapsed one-off buyers	40 %	£0

The ordering of the segments is exactly the ordering of their future value. Among discovery-year customers, Champions produced 66 % of next year's revenue and S2 21 %. Those returning customers produced 70 % of *all* next-year revenue (the rest came from new customers and anonymous invoices), so the Champions alone account for 47 % of everything the shop sold the next year. Reactivating "cooling" S2 customers (£1.37 M next-year revenue at stake) is worth more than chasing lapsed ones (£0.36 M).

But do they beat the standard tool? No. The pre-set pass mark is **not met**. Scored by how well the order of the groups ranks next-year activity:

Segmentation	AUC	Spearman ρ with next-year revenue
RFM-score quartiles (sum of R, F and M percentile ranks)	**0.765**	0.601
K-means segments (this project)	0.739	0.575
Monetary tertiles	0.729	0.586
Continuous RFM score (an upper bound)	0.783	0.628

The clusters predict better than spend alone, but not better than a plain RFM score. An alternative clustering on recency + one volume factor (the Stage 3 caveat) does no better (0.745) and agrees with the main one at ARI 0.73. **Conclusion:** the segments are valid as *descriptions*. They are stable, they order next-year value correctly, and each maps to a distinct action. For *ranking* customers within a campaign, the RFM score is the better tool. The app therefore pairs each customer's segment with the recency, frequency and spend values behind it.

5.3 Are the rules valid? Re-measure each rule on the following year

In [17]:
```
val_baskets = basket_matrix(val_lines, items=frequent_items)
def rule_stats(baskets, a, c):
    has_a = baskets[list(a)].all(axis=1); has_c = baskets[list(c)].all(axis=1)
    sup_a, sup_c, sup_ac = has_a.mean(), has_c.mean(), (has_a & has_c).mean()
    conf = sup_ac / sup_a if sup_a else np.nan
    return pd.Series({"val_support": sup_ac, "val_confidence": conf, "val_lift":
add_validation = lambda r: r.join(r.apply(lambda x: rule_stats(val_baskets, x["a
```

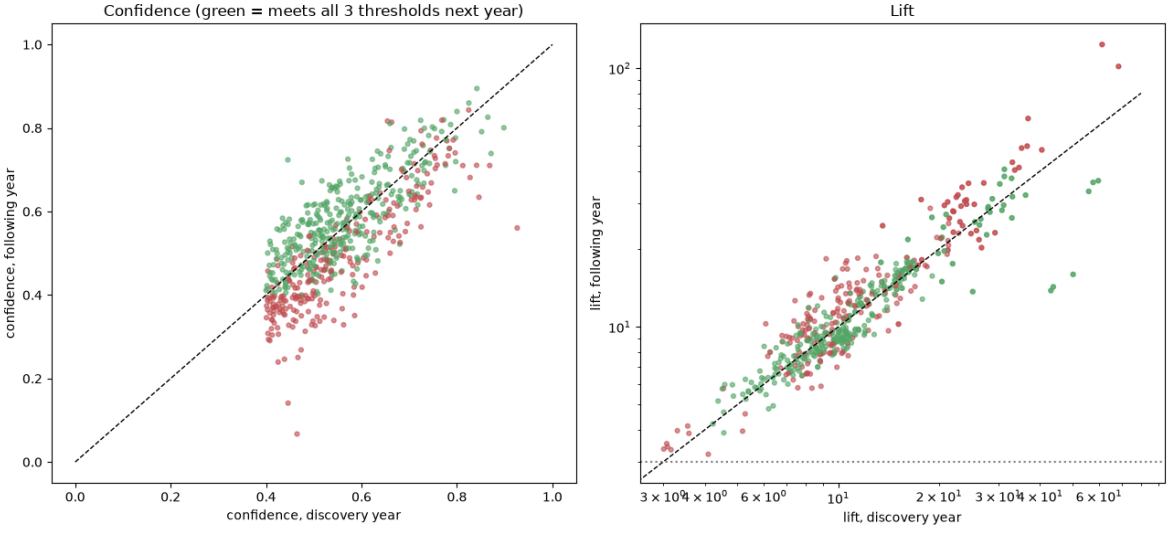
```
rules = add_validation(rules)
# A rule HOLDS next year only if it meets ALL THREE discovery thresholds again:
rules["holds_strict"] = (rules["val_support"] >= MIN_SUPPORT) & (rules["val_conf
rules["holds_conf_lift_only"] = (rules["val_confidence"] >= MIN_CONFIDENCE) & (r

# Persistence baseline: candidate rules just BELOW the thresholds in the discove
# often, persistence says little; if far less often, passing the thresholds real
_, near = mine_rules(disc_baskets, min_conf=0.25, min_lift=1.5)
near = near[~((near["confidence"] >= MIN_CONFIDENCE) & (near["lift"] >= MIN_LIFT
near = add_validation(near)
near["holds_strict"] = (near["val_support"] >= MIN_SUPPORT) & (near["val_confide
persist = pd.DataFrame({"rules": [len(rules), len(near)],
                        "meet all three thresholds next year %": [rules["holds_s
                        "meet confidence + lift only %": [rules["holds_conf_lift
                       index=["rules that passed the thresholds", "baseline: nea
persist.to_csv(REP / "rule_persistence.csv"); display(persist)
lift_ratio = (rules["val_lift"] / rules["lift"]).describe(percentiles=[.1, .5, .
print("Next-year lift / discovery lift:", lift_ratio[["10%", "50%", "90%"]].to_d

fig, axes = plt.subplots(1, 2, figsize=(13, 6))
col = rules["holds_strict"].map({True: "#55A868", False: "#C44E52"})
axes[0].scatter(rules["confidence"], rules["val_confidence"], c=col, s=12, alpha
axes[0].set(xlabel="confidence, discovery year", ylabel="confidence, following y
axes[1].scatter(rules["lift"], rules["val_lift"], c=col, s=12, alpha=.6); axes[1
axes[1].axhline(MIN_LIFT, c="gray", ls=":"); axes[1].set(xscale="log", yscale="l
savefig("evaluation_rule_validity")
```

	rules	meet all three thresholds next year %	meet confidence + lift only %
rules that passed the thresholds	598	58.9	85.3
baseline: near-miss rules (conf >= 0.25, lift >= 1.5)	516	11.8	NaN

```
Next-year lift / discovery lift: {'10%': 0.82, '50%': 1.03, '90%': 1.34}
```



In [18]:
```
# Are the thresholds a sensible trade-off? Mine with other support / lift settin
# and report how many rules survive and how much of the catalogue and of next ye
val_l = val_lines.assign(item=val_lines["StockCode"].map(item_name))
val_items_by_inv = val_l.groupby("Invoice")["item"].agg(set)
grid_rows = []
for sup in [0.005, 0.01, 0.02]:
```

```
    items_s = disc_support[disc_support >= sup].index
    b_disc, b_val = basket_matrix(disc_lines, items_s), basket_matrix(val_lines,
    _, r_all = mine_rules(b_disc, min_support=sup, min_lift=2.0)
    for lift in [2.0, 3.0, 5.0]:
        r = r_all[r_all["lift"] >= lift]
        if r.empty: continue
        st = r.apply(lambda x: rule_stats(b_val, x["antecedents"], x["consequent
        hold = (st["val_support"] >= sup) & (st["val_confidence"] >= MIN_CONFIDE
        kept = r[hold.to_numpy()]
        ants = {a for t in kept["antecedents"] for a in t}
        fires = np.zeros(len(b_val), bool)
        for a, c in zip(kept["antecedents"], kept["consequents"]):
            fires |= b_val[list(a)].all(axis=1).to_numpy() & ~b_val[list(c)].all
        grid_rows.append({"min_support": sup, "min_lift": lift, "rules": len(r),
                          "hold_next_year_%": hold.mean() * 100, "products_with_
                          "next_year_baskets_where_a_rule_fires_%": fires.mean()
threshold_grid = pd.DataFrame(grid_rows).round({"hold_next_year_%": 1, "next_yea
threshold_grid.to_csv(REP / "rule_threshold_sensitivity.csv", index=False); disp

# Sensitivity: anonymous invoices are much larger (2.4) - do they drive the rule
id_lines = disc_lines[disc_lines["Customer ID"].notna()]
id_support = item_support(id_lines)
_, id_rules = mine_rules(basket_matrix(id_lines, id_support[id_support >= MIN_SU
main_pairs, id_pairs = set(zip(rules["antecedents"], rules["consequents"])), set
print(f"Customer-ID invoices only: {len(id_rules)} rules vs {len(rules)}; {len(m
      f"are found again, and {len(main_pairs & id_pairs) / len(id_pairs):.0%} of
```

	min_support	min_lift	rules	itemsets	hold_next_year_%	products_with_a_validated_rul
0	0.005	2.0	4465	2373	46.8	30
1	0.005	3.0	4372	2312	47.6	29
2	0.005	5.0	4155	2191	48.6	28
3	0.010	2.0	616	342	57.5	11
4	0.010	3.0	598	325	58.9	10
5	0.010	5.0	581	313	58.9	10
6	0.020	2.0	65	46	53.8	2
7	0.020	3.0	62	43	56.5	2
8	0.020	5.0	60	41	55.0	2

```
Customer-ID invoices only: 429 rules vs 598; 65% of the main rules are found agai
n, and 90% of the ID-only rules are in the main set
```

Do the rules hold next year? A rule holds only if it meets **all three** discovery thresholds again: support ≥ 1 %, confidence ≥ 0.4 and lift ≥ 3. The earlier versions checked only confidence and lift, which gave 86 %.

- **59 %** of the 598 rules hold. 85 % meet confidence and lift alone; most of the rest fall just below 1 % support in the second year.
- **Baseline:** of 516 near-miss rules (confidence ≥ 0.25, lift ≥ 1.5, but below the thresholds in the discovery year), only **12 %** reach the thresholds the next year. Passing the thresholds therefore really selects stable patterns. The pass mark (≥ 50

%, clearly above the baseline) is met. It was set after an earlier review had estimated about this rate, so it is a documented target rather than a blind one.
- **Strength holds too:** next-year lift is typically 1.03 × the discovery lift (10th–90th percentile 0.82–1.34).

Threshold sensitivity (table above). This is *sensitivity*, not selection: the 1 % / 0.4 / 3 thresholds were fixed before mining, and the grid is scored on the validation year, so choosing from it would reuse that year. Confidence (0.4) and the 3-item limit were not varied.

- Lift barely matters, because at 40 % confidence low-support items already have high lift. Support is the real trade-off.
- At **0.5 %** there are 4,465 rules, only 47 % hold, 301 products get a validated rule, and a validated rule *fires* in 72 % of next-year baskets (all of its "if" products present, its "then" product absent).
- At **1 %** (used) 59 % hold, 109 products are covered, and a rule fires in 59 % of baskets.
- At **2 %** only 62 rules and 24 products remain (31 % of baskets).

Anonymous baskets: re-mining on invoices **with** a Customer ID only gives 429 rules, and 90 % of them are in the main set. The large anonymous invoices add rules (35 % of the main rules need them to reach 1 % support) but do not create different ones.

5.4 Are the rules novel? Obvious variants vs cross-category associations

In [19]:
```
# Novelty: a rule is an obvious VARIANT if its products are the same product typ
# or size (e.g. JUMBO BAG OWLS vs JUMBO STORAGE BAG SUKI); otherwise it links di
STYLE = r"\b(RED|PINK|BLUE|GREEN|WHITE|BLACK|IVORY|CREAM|PURPLE|YELLOW|ORANGE|SI
# product-type nouns: two products that share one of these are the same kind of
TYPES = {"BAG", "CASE", "PLATE", "CUP", "BOWL", "NAPKIN", "MUG", "HEART", "LIGHT
         "CLOCK", "SIGN", "FRAME", "CARD", "WRAP", "PEG", "DECORATION", "BUNTING
         "DOORMAT", "JAR", "TEACUP", "TRAY", "STICKER", "NOTEBOOK", "PENCIL", "C
         "COASTER", "DRAWER", "KNOB", "BELL", "BUCKET", "BANNER", "GARLAND", "HA
         "MAGNET", "PLAYHOUSE", "BALLOON", "STAND", "WALLET", "STRAW", "SPOON",
words = lambda name: re.findall(r"[A-Z]{2,}", re.sub(STYLE, " ", str(name).upper
types_of = lambda name: {w[:-1] if w.endswith("S") and w[:-1] in TYPES else w fo
def same_product(a, b):
    if types_of(a) & types_of(b):
        return True
    wa, wb = words(a), words(b)
    if len(wa) >= 2 and len(wb) >= 2 and (wa[:2] == wb[:2] or wa[-2:] == wb[-2:]
        return True
    sa, sb = set(wa), set(wb)
    return len(sa & sb) / max(len(sa | sb), 1) >= 0.5

from itertools import combinations
classify = lambda a, c: ("variant (same product, other design)" if all(same_prod
                         else "cross-product")
rules["rule_type"] = [classify(a, c) for a, c in zip(rules["antecedents"], rules
display(rules.groupby("rule_type").agg(rules=("lift", "size"), itemsets=("itemse
                                       holds_strict_pct=("holds_strict", lambda
```

```
show = ["antecedents", "consequents", "support", "confidence", "lift", "val_conf
print("Strongest cross-product rules that met all three thresholds again in the
cross = rules[(rules["rule_type"] == "cross-product") & rules["holds_strict"]].s
display(cross[show].head(12).round(3))
rules.to_csv(REP / "association_rules_evaluated.csv", index=False)
```

rule_type	rules	itemsets	median_lift	holds_strict_pct
cross-product	101	62	10.637144	33.7
variant (same product, other design)	497	263	10.888841	64.0

```
Strongest cross-product rules that met all three thresholds again in the followin
g year (one direction per pair):
```

	antecedents	consequents	support	confidence	lift	val_confidence	val_lift
22	(SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED SPOTTY PA...	(SET/6 RED SPOTTY PAPER PLATES,)	0.011	0.842	36.208	0.895	32.285
178	(CHARLOTTE BAG PINK POLKADOT, PACK OF 72 RETROSPOT CAKE ...	(RED RETROSPOT CHARLOTTE BAG,)	0.012	0.800	15.724	0.840	15.847
316	(PAINTED METAL PEARS ASSORTED,)	(ASSORTED COLOUR BIRD ORNAMENT,)	0.014	0.731	10.637	0.698	9.360
42	(SET/6 RED SPOTTY PAPER CUPS,)	(SET/6 RED SPOTTY PAPER PLATES,)	0.015	0.724	31.143	0.818	29.536
223	(CHARLOTTE BAG SUKI DESIGN, PACK OF 72 RETROSPOT CAKE CA...	(RED RETROSPOT CHARLOTTE BAG,)	0.010	0.697	13.697	0.762	14.375
264	(SET/6 RED SPOTTY PAPER PLATES,)	(SET/20 RED RETROSPOT PAPER NAPKINS,)	0.016	0.686	11.863	0.703	13.697
252	(WHITE HANGING HEART T-LIGHT HOLDER, WOODEN PICTURE FRAM...	(WOODEN FRAME ANTIQUE WHITE,)	0.015	0.680	12.307	0.522	10.592
545	(JUMBO BAG PINK POLKADOT, JUMBO SHOPPER VINTAGE RED PAIS...	(JUMBO BAG RED RETROSPOT,)	0.013	0.660	6.738	0.811	7.538

	antecedents	consequents	support	confidence	lift	val_confidence	val_lift
552	(JUMBO SHOPPER VINTAGE RED PAISLEY, JUMBO STORAGE BAG SUKI)	(JUMBO BAG RED RETROSPOT,)	0.012	0.633	6.471	0.745	6.923
298	(SET/6 RED SPOTTY PAPER CUPS,)	(SET/20 RED RETROSPOT PAPER NAPKINS,)	0.013	0.627	10.838	0.693	13.509
555	(GREEN REGENCY TEACUP AND SAUCER,)	(REGENCY CAKESTAND 3 TIER,)	0.011	0.612	6.393	0.496	4.946
559	(ROSES REGENCY TEACUP AND SAUCER,)	(REGENCY CAKESTAND 3 TIER,)	0.011	0.594	6.199	0.483	4.816

In [20]:
```
# Audit of the novelty classifier: a random sample of rules (one direction per i
# Hand rule: "variant" = the products are the same kind of item in another colou
# shopper are both large bags); "cross-product" = at least two different kinds o
audit = pd.concat([g.sample(min(20, len(g)), random_state=RANDOM_STATE) for _, g
audit = audit.assign(rule=audit["antecedents"].apply(" + ".join) + "  ->  " + au
HAND_LABELS = ["variant", "cross-product", "cross-product", "cross-product", "cr
audit["hand_label"] = HAND_LABELS
audit["classifier"] = audit["rule_type"].str.split(" ").str[0]
accuracy = (audit["classifier"] == audit["hand_label"]).groupby(audit["classifie
pd.set_option("display.max_colwidth", 110); display(audit[["rule", "classifier",
def proportion_confint(k, n, z=1.96):   # Wilson score interval
    p = k / n; c = (p + z * z / (2 * n)) / (1 + z * z / n); h = z * np.sqrt(p *
    return c - h, c + h
acc_tab = pd.DataFrame({"correct": (audit["classifier"] == audit["hand_label"]).
                        "sampled": audit.groupby("classifier").size()})
acc_tab["share_correct"] = acc_tab["correct"] / acc_tab["sampled"]
acc_tab[["ci_95_low", "ci_95_high"]] = [proportion_confint(c, n) for c, n in zip
display(acc_tab.round(2))
n_types = rules.drop_duplicates("itemset")["rule_type"].str.split(" ").str[0].va
est_cross = n_types["cross-product"] * accuracy["cross-product"] + n_types["vari
print(f"Estimated genuinely cross-product itemsets: about {est_cross:.0f} of {n_
      f"correct calls plus variants it misfiled). The hand labels are the author
held = rules.drop_duplicates("itemset").groupby("rule_type")["holds_strict"].mea
print("Share of itemsets whose rule holds next year, by type:", held.round(3).to
audit[["rule", "classifier", "hand_label"]].to_csv(REP / "novelty_audit.csv", in
```

	rule	classifier	hand_label
474	JUMBO BAG SPACEBOY DESIGN -> JUMBO SHOPPER VINTAGE RED PAISLEY	cross-product	variant
561	MINI CAKE STAND HANGING STRAWBERY -> STRAWBERRY CERAMIC TRINKET BOX	cross-product	cross-product
22	SET/20 RED RETROSPOT PAPER NAPKINS + SET/6 RED SPOTTY PAPER CUPS -> SET/6 RED SPOTTY PAPER PLATES	cross-product	cross-product
563	CERAMIC STRAWBERRY CAKE MONEY BANK -> STRAWBERRY CERAMIC TRINKET BOX	cross-product	cross-product
100	PINK POLKADOT BOWL -> PINK POLKADOT CUP	cross-product	cross-product
471	60 TEATIME FAIRY CAKE CASES + JUMBO BAG RED RETROSPOT -> PACK OF 72 RETROSPOT CAKE CASES	cross-product	cross-product
218	3 STRIPEY MICE FELTCRAFT -> FELTCRAFT BUTTERFLY HEARTS	cross-product	variant
178	CHARLOTTE BAG PINK POLKADOT + PACK OF 72 RETROSPOT CAKE CASES -> RED RETROSPOT CHARLOTTE BAG	cross-product	cross-product
291	FELTCRAFT BUTTERFLY HEARTS -> FELTCRAFT 6 FLOWER FRIENDS	cross-product	variant
580	SET OF 72 PINK HEART PAPER DOILIES -> PACK OF 72 RETROSPOT CAKE CASES	cross-product	cross-product
323	JUMBO SHOPPER VINTAGE RED PAISLEY + JUMBO STORAGE BAG SUKI -> JUMBO BAG PINK POLKADOT	cross-product	variant
147	RED RETROSPOT SMALL MILK JUG -> RED RETROSPOT SUGAR JAM BOWL	cross-product	cross-product
463	PINK DOUGHNUT TRINKET POT -> STRAWBERRY CERAMIC TRINKET BOX	cross-product	variant
186	WOOD S/3 CABINET ANT WHITE FINISH + WOODEN PICTURE FRAME WHITE FINISH -> WOODEN FRAME ANTIQUE WHITE	cross-product	cross-product
368	WOOD 2 DRAWER CABINET WHITE FINISH -> WOODEN PICTURE FRAME WHITE FINISH	cross-product	cross-product
372	BAKING SET SPACEBOY DESIGN -> BAKING SET 9 PIECE RETROSPOT	cross-product	variant
85	BLUE POLKADOT BOWL -> BLUE POLKADOT CUP	cross-product	cross-product
223	CHARLOTTE BAG SUKI DESIGN + PACK OF 72 RETROSPOT CAKE CASES -> RED RETROSPOT CHARLOTTE BAG	cross-product	cross-product
556	SET OF 72 RETROSPOT PAPER DOILIES -> PACK OF 72 RETROSPOT CAKE CASES	cross-product	cross-product
145	RED RETROSPOT BOWL -> RED RETROSPOT CUP	cross-product	cross-product
263	72 SWEETHEART FAIRY CAKE CASES + PACK OF 72 RETROSPOT CAKE CASES -> PACK OF 60 DINOSAUR CAKE	variant	variant

	rule	classifier	hand_label
	CASES		
274	60 TEATIME FAIRY CAKE CASES + PACK OF 60 PINK PAISLEY CAKE CASES -> PACK OF 60 DINOSAUR CAKE CASES	variant	variant
593	FOLKART ZINC HEART CHRISTMAS DEC -> WHITE HANGING HEART T-LIGHT HOLDER	variant	cross-product
531	LUNCH BAG PINK POLKADOT -> LUNCH BAG RED RETROSPOT	variant	variant
151	WOOD S/3 CABINET ANT WHITE FINISH -> WOOD 2 DRAWER CABINET WHITE FINISH	variant	variant
575	JUMBO BAG PINK VINTAGE PAISLEY -> JUMBO BAG RED RETROSPOT	variant	variant
396	JUMBO BAG PINK POLKADOT + JUMBO BAG RED RETROSPOT -> JUMBO BAG STRAWBERRY	variant	variant
504	PACK OF 60 SPACEBOY CAKE CASES -> PACK OF 60 PINK PAISLEY CAKE CASES	variant	variant
18	COFFEE MUG DOG + BALL DESIGN -> COFFEE MUG CAT + BIRD DESIGN	variant	variant
233	60 TEATIME FAIRY CAKE CASES + 72 SWEETHEART FAIRY CAKE CASES -> PACK OF 60 DINOSAUR CAKE CASES	variant	variant
55	BLUE SPOT CERAMIC DRAWER KNOB -> RED SPOT CERAMIC DRAWER KNOB	variant	variant
257	PACK OF 72 RETROSPOT CAKE CASES + PACK OF 72 SKULL CAKE CASES -> PACK OF 60 DINOSAUR CAKE CASES	variant	variant
108	HAND WARMER OWL DESIGN -> HAND WARMER BIRD DESIGN	variant	variant
523	LUNCH BAG WOODLAND -> LUNCH BAG RED RETROSPOT	variant	variant
438	JUMBO BAG RED RETROSPOT + JUMBO STORAGE BAG SUKI -> JUMBO BAG BAROQUE BLACK WHITE	variant	variant
503	LUNCH BAG WOODLAND -> LUNCH BAG BLACK SKULL.	variant	variant
176	RED RETROSPOT CHARLOTTE BAG + WOODLAND CHARLOTTE BAG -> CHARLOTTE BAG PINK POLKADOT	variant	variant
12	PINK REGENCY TEACUP AND SAUCER -> ROSES REGENCY TEACUP AND SAUCER	variant	variant
588	RECYCLING BAG RETROSPOT -> JUMBO BAG RED RETROSPOT	variant	variant
153	JUMBO STORAGE BAG SUKI + RED RETROSPOT CHARLOTTE BAG -> CHARLOTTE BAG SUKI DESIGN	variant	variant

classifier	correct	sampled	share_correct	ci_95_low	ci_95_high
cross-product	14	20	0.70	0.48	0.85
variant	19	20	0.95	0.76	0.99

```
Estimated genuinely cross-product itemsets: about 57 of 325 (classifier says 62;
correct calls plus variants it misfiled). The hand labels are the author's own, s
o treat this as indicative.
Share of itemsets whose rule holds next year, by type: {'cross-product': 0.274,
'variant (same product, other design)': 0.624}
```

Evaluation against the Stage 0 criteria

Criterion	Evidence	Verdict
Valid	**Segments:** next-year activity 96 % / 74 % / 65 % / 40 % in the predicted order; out-of-bag bootstrap ARI 0.92. But a plain RFM score ranks next-year activity better (AUC 0.765 vs 0.739, gap CI +0.018 to +0.034). **Rules:** 59 % meet all three thresholds again next year, against 12 % of near-miss rules; median lift changes by only +3 %.	✅ rules · ⚠️ segments: valid as descriptions, beaten as predictors
Novel	263 of the 325 itemsets are classified as variants (same product, other colour/design): expected, but useful for "complete the set" prompts. A hand audit of 40 sampled itemsets, labelled by the author (so indicative only), shows the classifier is right for 95 % of variants (CI 76–99 %) and **70 %** of cross-product calls (CI 48–85 %). Counting cross-product itemsets misfiled as variants too, about **57 of the 325** are genuine cross-sells. These are customers buying **matching-design sets across product types**: red spotty napkins + cups → plates, Regency teacup and saucer → the matching cake stand, painted metal pears → bird ornament. **But only 27 % of cross-product itemsets hold next year** (variants: 62 %), so as a class the cross-sells are much less stable than the variants. Only the strongest ones (top of the table above) are reliable. Much of this follows the retailer's own collections, so it is better called a *confirmation with numbers* than a surprise.	⚠️ partly: novel patterns exist but are mostly unstable
Potentially useful	Each segment has an action. S2 customers went on to spend £1.37 M the next year: the revenue a re-engagement campaign aims to keep, not money already lost. 352 validated rules (34 cross-product) cover 109 products, and a validated rule fires in 59 % of next-year baskets. That is only 109 of 4,047 products, which bounds the recommender's reach.	✅ with limited coverage
Understandable	Segments described in plain words with member shares; rules shown as product names with confidence and lift.	✅

Iterations made (KDD loops back):

- **Round 1:** the first evaluation used median-split labels (mislabelling S3 as "frequent"), lenient validity thresholds (99 % "held") and a word-matching novelty test that called

key-fob variants cross-category.
- **Round 2:** an independent review found that cleaning leaked across the split, `return_rate` double-counted, the "full thresholds" test skipped support (86 % → 59 %), the segments had no baseline, and the deployed app mislabelled its segments.
- **Round 3:** the two Excel sheets overlapped (22,523 lines loaded twice), and the coverage figure counted rules that did not fully fire.

All were corrected, and Stage 5 now reports one criterion as not met and one as only partly met. See `critiques/round_1.md` – `round_3.md`.

🔍 Critic review — Interpretation and evaluation

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Interpretation and evaluation** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the patterns were genuinely tested on data not used to find them, whether novelty and usefulness are argued rather than asserted, and whether loops back to earlier stages were considered. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Median-split labels mislabel S3; validity thresholds halved (too lenient); novelty test calls key-fob variants cross-category.	Tertile labels + 'new' flag; strict validity test (86 % hold); product-type novelty test; app uses only strictly validated rules.
2	"86 % hold the full thresholds" skipped support (really 59 %); no segment baseline; novelty classifier unaudited.	Strict three-threshold test + near-miss baseline (12 %); RFM baselines (clusters lose: 0.739 vs 0.765); hand audit.
3	Cross-product rules hold only ~27–34 % of the time; audit without CIs; coverage counted rules that did not fire.	Claims rewritten (novel but mostly unstable); Wilson CIs, ≈ 57 genuine cross-sells; coverage = rules that fire (59 %).

Step 9 — Acting on the discovered knowledge

Fayyad step 9: use the knowledge directly, incorporate it into another system, or document and report it.

- **Segments:** the app ships the **validated** discovery-year segmenter (the same model, the same 12-month window). It scores every customer on their most recent 12 months, so what was tested in 5.2 is what staff see.
- **Rules:** re-mined on both years for current products. Each is flagged if the same rule was validated, and the app shows only validated rules by default.
- **Label check:** the cell below re-profiles the deployed segments with the discovery cut-offs, to confirm each still matches its name.

In [21]:
```
import joblib
# Deploy the VALIDATED segmentation: the discovery-year model (12-month RFM wind
# recent 12 months. Same window length, same model, same names - so what was val
snapshot = df["InvoiceDate"].max().normalize() + pd.Timedelta(days=1)
window_start = snapshot - pd.Timedelta(days=365)
recent = df[df["InvoiceDate"] >= window_start]
final_customers = customer_features(recent, returns[returns["InvoiceDate"] >= wi
final_customers["cluster"] = segmenter.predict(final_customers[SEGMENT_FEATURES]
final_customers["segment"] = final_customers["cluster"].map(SEGMENT_OF)
final_customers["rfm_score_pct"] = ((final_customers["recency_days"].rank(ascend
                                     + final_customers["monetary"].rank(pct=True)

# Label check: do the deployed segments still look like their names? Profiles us
final_prof = profile_segments(final_customers).rename(index=SEGMENT_OF).sort_ind
label_words = lambda s: [w for w in s.split() if not w.endswith("%") and w != "·
check = pd.DataFrame({"discovery_profile": prof["profile"], "deployed_profile":
                      "discovery_customers": prof["customers"], "deployed_custom
                      "deployed_median_recency": final_prof["recency_days"], "de
check["labels_agree"] = [label_words(a)[:3] == label_words(b)[:3] for a, b in zi
check.to_csv(REP / "deployed_segment_label_check.csv"); display(check)
print("All deployed segments match their names." if check["labels_agree"].all()

# Rules for the app: re-mined on both years (more data, current products) and fl
full_lines = df[df["period"] != "after"]
full_support = item_support(full_lines)
_, final_rules = mine_rules(basket_matrix(full_lines, full_support[full_support
validated = set(zip(rules.loc[rules["holds_strict"], "antecedents"], rules.loc[r
final_rules["validated_next_year"] = [(a, c) in validated for a, c in zip(final_
final_rules["rule_type"] = [classify(a, c) for a, c in zip(final_rules["antecede
out_rules = final_rules.assign(antecedent=final_rules["antecedents"].apply(" + "
    ["antecedent", "consequent", "support", "confidence", "lift", "validated_nex
out_rules.to_csv(APP / "rules.csv", index=False)
final_customers.reset_index().to_csv(APP / "customer_segments.csv", index=False)
joblib.dump(segmenter, APP / "segmenter.joblib")
meta = {"created": datetime.now().isoformat(timespec="seconds"), "k": K, "segmen
        "snapshot": str(snapshot.date()), "window": f"{window_start.date()} .. {
        "segments": SEGMENT_ACTIONS, "cluster_to_segment": {str(k): v for k, v i
        "profiles": final_prof.drop(columns="revenue").reset_index(names="segmen
        "versions": {"python": platform.python_version(), "numpy": np.__version_
(APP / "metadata.json").write_text(json.dumps(meta, indent=2, default=str), enco
print(f"{len(final_customers):,} customers active in the last 12 months segmente
      f"({out_rules['validated_next_year'].sum():,} validated, {(out_rules['vali
      f"| {len({p for a in out_rules.loc[out_rules['validated_next_year'], 'ante
display(final_customers["segment"].value_counts().sort_index().to_frame("custome
```

	discovery_profile	deployed_profile	discovery_customers	deployed_customers	deploye
S1	active 85% · frequent 98% · high-spend 96%	active 88% · frequent 97% · high-spend 97%	712	682	
S2	cooling 60% · repeat 67% · high-spend 55%	cooling 60% · repeat 67% · high-spend 52%	1189	1176	
S3	active 83% · repeat 63% · mid-spend 57% · new 56%	active 80% · repeat 67% · mid-spend 55% · new 56%	832	784	
S4	lapsed 70% · one-off 75% · low-spend 74%	lapsed 68% · one-off 77% · low-spend 72%	1497	1608	

```
All deployed segments match their names.
4,250 customers active in the last 12 months segmented | 639 rules (352 validate
d, 34 of them cross-product) | 109 products have a validated rule
```

segment	customers
S1	682
S2	1176
S3	784
S4	1608

In [22]:
```
APP_CODE = r"""
import json
from pathlib import Path
import joblib
import pandas as pd
import streamlit as st

HERE = Path(__file__).parent
st.set_page_config(page_title="Retail insights", page_icon="🛍️")

@st.cache_resource
def load():
    meta = json.loads((HERE / "metadata.json").read_text(encoding="utf-8"))
    rules = pd.read_csv(HERE / "rules.csv")
    rules["ante_set"] = rules["antecedent"].str.split(" + ", regex=False).apply(
    rules["cons_set"] = rules["consequent"].str.split(" + ", regex=False).apply(
    customers = pd.read_csv(HERE / "customer_segments.csv", dtype={"Customer ID"
    return meta, rules, customers, joblib.load(HERE / "segmenter.joblib")

meta, rules, customers, segmenter = load()
st.title("🛍️ Customer segments & product recommendations")
st.caption(f"Online Retail II · {len(customers):,} customers active in {meta['wi
seg_info = meta["segments"]

recs, lookup, new_cust, overview = st.tabs(["Product recommendations", "Customer
with recs:
```

```
    only_valid = st.checkbox("Only rules that met support, confidence and lift a
    hide_variants = st.checkbox("Hide colour/design variants of the same product
    pool = rules[rules["validated_next_year"]] if only_valid else rules
    if hide_variants:
        pool = pool[pool["rule_type"] == "cross-product"]
    products = sorted({p for s in pool["ante_set"] for p in s})
    basket = set(st.multiselect("Products in the customer's basket", products, d
    hits = pool[pool["ante_set"].apply(lambda a: a <= basket) & pool["cons_set"]
    if not basket:
        st.info("Pick one or more products.")
    elif hits.empty:
        st.info("No association for this basket - every rule needs ALL of its 'i
    else:
        st.dataframe(hits[["antecedent", "consequent", "confidence", "lift", "ru
        top = hits.iloc[0]
        st.success(f"Suggest **{top['consequent']}**: bought in {top['confidence
                   f"{top['antecedent']} ({top['lift']:.1f}x more often than ave
with lookup:
    cid = st.text_input("Customer ID", value=customers["Customer ID"].iloc[0])
    row = customers[customers["Customer ID"] == cid.strip()]
    if row.empty:
        st.warning("Customer not found (only customers who ordered in the last 1
    else:
        r = row.iloc[0]; s = seg_info[r["segment"]]
        st.subheader(s["name"]); st.write(f"**Recommended action:** {s['action']
        c1, c2, c3 = st.columns(3)
        c1.metric("Days since last order", int(r["recency_days"])); c2.metric("O
        st.write(f"**RFM score:** better than {int(r['rfm_score_pct'])} % of cus
                 "(it predicts next-year activity slightly better than the segme
with new_cust:
    st.write("Enter the customer's activity over the **last 12 months** (the win
    c1, c2, c3 = st.columns(3)
    rec = c1.number_input("Days since last order", 0, 365, 30)
    freq = c2.number_input("Orders in the last 12 months", 1, 500, 3)
    mon = c3.number_input("Spend in the last 12 months (GBP)", 1.0, 1e6, 500.0)
    cluster = int(segmenter.predict(pd.DataFrame([{"recency_days": rec, "frequen
    seg = meta["cluster_to_segment"][str(cluster)]
    st.subheader(seg_info[seg]["name"]); st.write(f"**Recommended action:** {seg
with overview:
    st.write(f"Segment profiles for customers active in {meta['window']} (median
    st.dataframe(pd.DataFrame(meta["profiles"]), hide_index=True)
    pick = st.selectbox("Export a segment's customer list (ranked by RFM score)"
    lst = customers[customers["segment"] == pick].sort_values("rfm_score_pct", a
    st.download_button(f"Download {pick} ({len(lst):,} customers)", lst.to_csv(i
"""
(APP / "app.py").write_text(APP_CODE.strip() + "\n", encoding="utf-8")
v = meta["versions"]
(APP / "requirements.txt").write_text(f"streamlit>=1.40\nscikit-learn=={v['sciki
compile(APP_CODE, "app.py", "exec")
print("App files:", sorted((p.name, f"{p.stat().st_size / 1e6:.1f} MB") for p in
print(f"When deploying, choose Python {v['python'][:4]} so the pinned versions i
```

```
App files: [('app.py', '0.0 MB'), ('customer_segments.csv', '0.3 MB'), ('metadat
a.json', '0.0 MB'), ('requirements.txt', '0.0 MB'), ('rules.csv', '0.1 MB'), ('se
gmenter.joblib', '0.0 MB')]
When deploying, choose Python 3.13 so the pinned versions install and the segment
er loads.
```

Deploy (free): put `artifacts/app/` in your GitHub repo → share.streamlit.io → *Create app* → `kdd/artifacts/app/app.py`; pick the Python version printed above. To change

segment names or actions, edit `SEGMENT_ACTIONS` and re-run.

What the app does:

- **Recommendations.** Staff enter the **whole basket**, and a rule fires only when *all* of its "if" products are in it, so the confidence shown applies to that basket. A checkbox hides colour/design variants to show cross-sells only.
- **Customer lookup.** Shows the customer's segment and action **plus an RFM score** (percentile), the better tool for ranking customers *within* a segment (5.2).
- **Segment a customer.** The inputs are explicitly *last-12-month* figures, the window the model was built and validated on.
- **Segments overview.** Shows the deployed segments' own profiles, and exports each segment's customer list ranked by RFM score for a campaign.
- **Label check (above).** All four deployed segments still match their names, with the same majority labels and the same order: S1 682, S2 1,176, S3 784 and S4 1,608 customers active in the last 12 months.

Consolidating the knowledge

- **Agreement with prior knowledge:** the segments match classic RFM intuition — a small group of recent, frequent buyers carries most revenue (17 % of customers → 64 % of revenue) — and an RFM score does better at prediction. This builds trust in the result and limits its novelty.
- **What is new:** some design collections are bought together *across* product types, strongly and stably (red spotty napkins + cups → plates: confidence 84 %, still 90 % the next year). For a giftware merchandiser this suggests organising the web shop by *collection* (all "red retrospot" items together) as well as by product type, and bundling matching items. Most cross-product associations, however, do not repeat the next year, so only the validated ones should be promoted.
- **Conflicts to watch:** most rules are colour/design variants; promoting them only helps if the product page doesn't already show all colours.
- **Measuring the effect:** run each segment action and the recommendation tab as an **A/B test** with a random 20 % hold-out per segment. Compare next-quarter purchase rate and basket size; this ties the result back to the Stage 0 goals.
- **Maintenance (owner: the marketing analyst):**
  - every quarter, re-mine on a **rolling 12-month window**;
  - validate each mined set on the **following 12 months** once they are available, the same design as this notebook, so validation lags the mining by a year;
  - in between, track monthly how often the published rules still meet their thresholds on the latest 12 months, and alert if that falls below 50 % or a segment's size moves by more than 25 %;
  - re-run the label check before publishing.
- **Data governance:** the app holds customer IDs with their spend. That is acceptable for this public research dataset, but a real deployment needs access control and a retention rule.

🔍 Critic review — Acting on the knowledge

Each stage was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the* ***Acting on the knowledge*** *stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the delivered outputs serve the end users identified in Stage 0, whether only validated patterns are promoted, and whether a refresh/maintenance plan exists. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Segment actions were placeholders; no refresh plan.	Named segments with concrete actions; quarterly re-mining recommended.
2	App shipped a refitted, mislabelled segmentation ("cooling" segment with 178-day recency).	App ships the validated 12-month segmenter on the last 12 months; label check passes; whole-basket recommendations.
3	RFM score (the better ranker) not in the app; no per-segment export; maintenance window inconsistent.	RFM score per customer and per-segment export in the app; 12-month lagged validation in the maintenance plan.